

HERON

社内機材管理システム
要件定義 兼 基本設計書

Document Version: 2.0 (Color Updated)

Target Environment: macOS / Docker / NIIMBOT
M2

Tech Stack: Angular / Go / Relational DB

第1部：要件定義書

1. システム概要と背景

アニメーションスタジオでは、液タブ（ペンディスプレイ）、高解像度モニター、PC、検証用タブレットなど、高価かつ専門的な機材が多数運用されている。これらの機材はプロジェクトや担当カットの変更に伴い、頻繁に社内を移動する。現状、誰がどの機材を使用しているか、あるいはどの部屋・棚に保管されているかの追跡が属人化・形骸化しやすく、「必要な時に機材が見つからない」「棚卸しに莫大な工数がかかる」といった課題がある。

本プロジェクト「HERON」は、この課題を解決するため、機材に固有のIDとQRコード付きラベルを発行し、スマートフォンやPCのブラウザから即座に貸出・返却・棚卸し・検索が行えるスマートなシステムを構築する。

2. ユーザー権限とロール

システムを利用するユーザーは大きく2つの権限に分類される。

権限	想定ユーザー	許可される操作
管理者 (Admin)	情シス部門、機材管理担当者	- 機材の新規登録、編集、削除 - ラベル(QR)の発行・印刷 - スマートフォン等を用いたスキャン（貸出・返却・棚卸し実行） - マスタデータ（部屋、棚情報）の管理 - 全利用者の履歴閲覧
一般 (General)	アニメーター、制作進行など 全社員	- 自身の現在貸出中の機材一覧の確認 - システム全体からの機材検索（現在のステータスと所在の確認のみ） ※貸出・返却のシステム操作は行えない

3. 業務・機能要件

3.1 機材IDと命名ルール

採用ルール: **HRN** - [カテゴリ(2~3文字)] - [連番4桁]

部署名や使用者名は変動するため物理ラベルには印字せず、DB上のメタデータとして扱う。

- **プレフィックス:** **HRN** (固定)
- **カテゴリ例:** **PC** (パソコン), **DSP** (ディスプレイ), **TAB** (ペンタブ・液タブ), **CAM** (カメラ)
- **連番:** **0001** からのゼロパディング。システムがカテゴリごとに自動採番を行う。
- **発番例:** **HRN-TAB-0108**

3.2 ステータス（状態）管理

機材は常に以下のいずれかの状態を持つ。

- **保管中:** 特定のロケーション（部屋・棚）に保管されており、即時貸出可能。
- **利用中:** 特定のユーザー（社員）に貸し出されている状態。
- **修理・メンテナンス中:** 故障や色校正のため、一時的に使用不可。
- **廃棄・除却:** すでに破棄済みだが、履歴を保持するためにデータ上残している状態。

3.3 QRスキャン・ラベル連携機能要件

- **単一スキャン（貸出 / 返却）:** 管理者がスマホのカメラで機材のQRをスキャン。画面に機材情報がポップアップし、その場で「誰に貸すか」を選択し貸出処理、または「返却」処理を1タップで完了させる。
- **連続スキャン（棚卸しモード）:** アプリ側で「対象の部屋・棚」を選択後、カメラモードを起動。次々にQRコードを読み込ませることで、対象機材の現在位置をその棚に上書きし、「保管中」ステータスとして棚卸しログを記録する。
- **ラベル印刷（NIIMBOT M2連携）:** Angularの画面から印刷用ビュー（HTML/CSS）を生成し、OS標準の印刷ダイアログ（window.print）経由でNIIMBOT M2へ出力する。

4. 非機能要件

- **UI/UX:** 管理者のスキャン操作はモバイル（スマホブラウザ）を想定。レスポンスデザインとし、カメラ起動の高速化・連続スキャン時の動作の軽快さを重視。
- **稼働環境:** DockerおよびDocker Composeを利用し、Macのローカル環境へ容易にテストデプロイ・開発ができること。
- **パフォーマンス:** QR読み取りから機材情報の表示までのAPIレスポンスは1秒以内を目標とする。

第2部：UI/UX デザイン仕様

1. コンセプトカラー（Color Palette）

プロジェクト「HERON」のUI/UXでは、アオサギの持つ「スマートさ（知性）」や「静寂（保管）」を表現するため、ネイビーとグレーを基調とした洗練されたモノトーン調のグラデーションを採用する。これにより、長時間の業務でも目が疲れにくく、かつプロフェッショナルな印象を与えるシステムを実現する。



Main 1 (Heron Navy)

#2A3A4A

テキスト、サイドバー、アイコン等。システム全体を引き締める色。



Gradient Step 1

#465463



Gradient Step 2

#626F7B



Gradient Step 3 (中間色)

#7F8994



Gradient Step 4

#9BA3AD



Gradient Step 5

#B7BEC5



Main 2 (Silver Plumage)

#D3D8DE

ベース背景、パネル、仕切り線など。メイン1とのコントラスト用。

グラデーション・レビュー (副色)

主要なボタンのホバーエフェクトやプログレスバー、ヘッダー背景等に使用し、UIに金属的な光沢感と奥行きをもたらす。



CSS実装例 (Angular / Tailwind等へ適用):

```
background: linear-gradient(135deg, #465463, #626F7B, #7F8994, #9BA3AD,  
#B7BEC5);
```

第3部：基本設計書

1. システムアーキテクチャ

最新のWeb標準技術とコンテナ技術を組み合わせ、保守性と開発速度を両立する。

- **フロントエンド**: Angular (TypeScript)
 - UIライブラリ: Angular Material または Tailwind CSS（上記コンセプトカラーをテーマとして設定）
 - QR読取: `ngx-scanner` または `zxing-js` ライブラリ
- **バックエンド**: Go言語 (Go Modules)
 - フレームワーク: `Gin` または `Echo`（軽量で高速なルーティング）
 - O/Rマッパー: `GORM`
- **データベース**: PostgreSQL 15+ または MySQL 8.x (Dockerコンテナ)

2. データベース設計（主要テーブル）

リレーショナルデータモデルを採用し、正規化によるデータ整合性を担保する。

2.1 Users テーブル（ユーザーマスタ）

カラム名	データ型	制約	備考
user_id	VARCHAR(36)	PK	UUIDを想定。
name	VARCHAR(100)	NOT NULL	社員の氏名
role	VARCHAR(20)	NOT NULL	'admin' または 'general'

2.2 Locations テーブル（保管場所マスタ）

カラム名	データ型	制約	備考
------	------	----	----

location_id	INT	PK, AUTO_INC	内部ID
room_name	VARCHAR(100)	NOT NULL	例: 第1作画室
shelf_name	VARCHAR(100)	NOT NULL	例: 機材棚A-3段目

2.3 Equipments テーブル（機材マスタ）

機材の「現在の最新状態」のみを管理する。

カラム名	データ型	制約	備考
equipment_id	VARCHAR(20)	PK	例: HRN-PC-0001
category	VARCHAR(10)	NOT NULL	PC, TAB, DSP 等
status	VARCHAR(20)	NOT NULL	available, in_use, maintenance, discarded
current_user_id	VARCHAR(36)	FK, NULL	利用中(in_use)の場合に設定
current_location_id	INT	FK, NULL	保管中の場合に設定

2.4 Transaction_Logs テーブル（履歴・監査ログ）

アニメ制作現場で重要な「いつ・誰が・どこへ動かしたか」の証跡を残す。

カラム名	データ型	制約	備考
log_id	BIGINT	PK, AUTO_INC	
equipment_id	VARCHAR(20)	FK, NOT NULL	対象の機材ID
action_type	VARCHAR(20)	NOT NULL	lend(貸出), return(返却), inventory(棚卸し), update(更新)
actor_user_id	VARCHAR(36)	FK, NOT NULL	操作を実行した管理者ID

target_user_id	VARCHAR(36)	FK, NULL	貸出先ユーザー（lend時のみ）
target_location_id	INT	FK, NULL	返却・棚卸し先のロケーションID
timestamp	TIMESTAMP	NOT NULL	操作日時

3. API仕様設計（Go言語によるREST API）

フロントエンドからバックエンドへ通信する際の主要なエンドポイント。

- **GET /api/equipments** : 機材一覧の取得（検索条件、ページネーション対応）
- **GET /api/equipments/{id}** : 機材詳細と最新の履歴数件を取得
- **POST /api/equipments** : 機材の新規登録・ID採番処理
- **POST /api/transactions/lend** : 貸出処理（対象機材ID、対象ユーザーIDを送信）
- **POST /api/transactions/return** : 返却処理（対象機材ID、返却先ロケーションIDを送信）
- **POST /api/transactions/inventory** : 一括棚卸し処理（ロケーションIDと、スキャンされた複数の機材IDリストを配列で送信）

4. 外部機器連携（NIIMBOT M2印刷機能）

NIIMBOT専用アプリを経由せずWebから直接印刷するための技術仕様。

印刷フローと実装方針

1. 開発用MacにNIIMBOT M2をペアリングし、汎用ドライバでプリンタとして認識させる。
2. Angular内に **/print/label/{equipment_id}** の隠しルートを作成する。
3. このルートでは **@media print** CSSを用いて、余白ゼロ、幅30mm・高さ15mm（利用するシールの実寸）の描画領域を厳密に定義する。
4. TypeScript（あるいはバックエンド）で生成したQRコード画像(Base64)と機材ID文字列を配置する。
5. UIの「印刷」ボタン押下時に **window.print()** をコールし、プリンタダイアログを起動。管理者がNIIMBOTを選択して出力する。